Enhancing Fantasy Point Prediction

DSCI 510 - Spring 2026 Final Project

Daniel Vasquez April 2026

Introduction

- Fantasy basketball requires predicting how many points a player will score in a single game before it happens

- This project builds a machine learning pipeline that predicts NBA fantasy points using two full seasons of player game data (2024-25 and 2025-26)

Research question:

- Do contextual factors - opponent defense, pace, home/away, injuries, and back to back games - improve prediction accuracy beyond player history alone?

Data Sources

Source	Collection Type	What it provides
nba_api	**Open source Python client library (JSON)**	Player game logs - PTS, REB, AST, STL, BLK, TO, 3pm, minutes (2 seasons ~530 players)
Baseketball Reference	**Web page (HTML) - Webscrape**	Team advanced stats - ORtg, DRtg, Pace per season (scraped)
Sleeper API	**API (JSON)**	Real-time injury status and notes per player
ESPN Injuries	**Web page (HTML) - Webscrape**	Injury validation - cross-checked against Sleeper coverage

- 48,874 game logs across 524 players

Note: No API keys required. All sources are open APIs or publicly accessible webpages.

Feature Engineering & Model Setup

- Calculate fantasy points per game using a generic fantasy scoring system
  - (PTS x 1) + (REB x 1.2) + (AST x 1.5) + (STL x 3) + (BLK x 3) + (TO x -1) + FG3M x 0.5

Features built:

- Exponentially Weighted Moving Averages (EWMA span=5 and span =10) for all major stats
- Opponent DRtg & Pace joined from Basketball Reference
- Own team ORtg, Home/Away flag, Injury flag, Back-to-back flag

Train/Test Split:

- Train: 2024-25 season
- Test: 2025-26 season

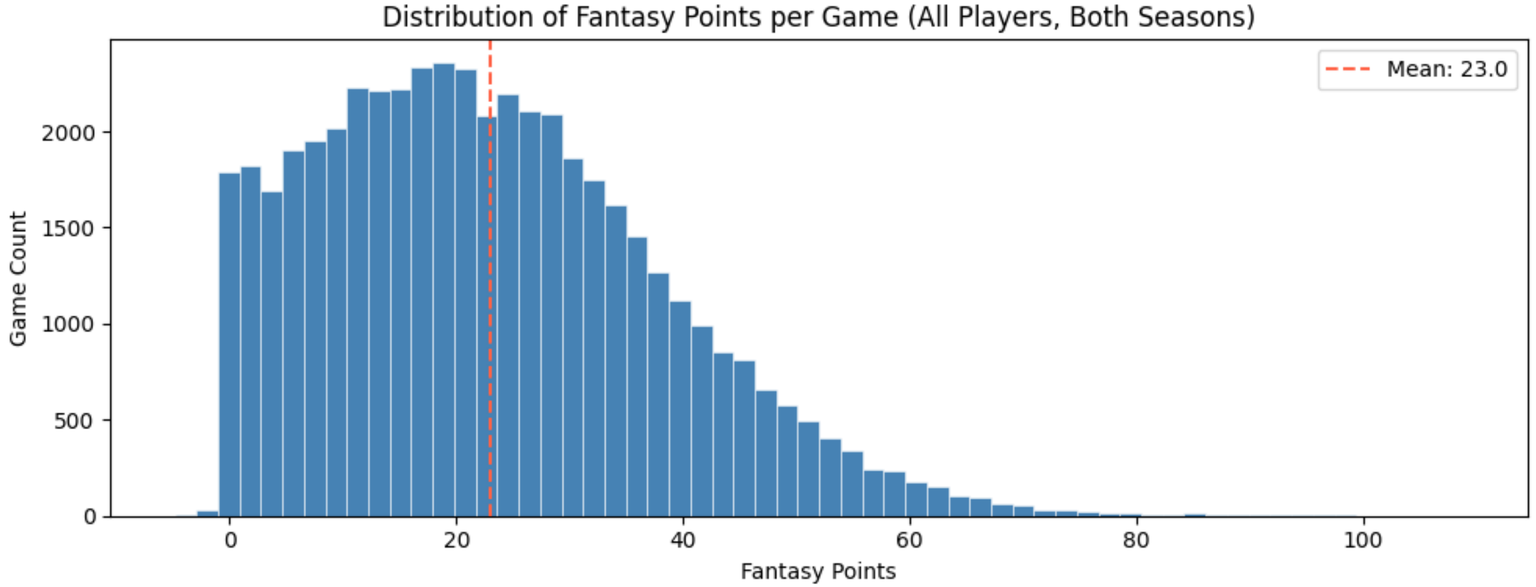
Feature Definitions

Feature	Description	How it was measured
FANTASY_PTS_EWMA5	Player's recent fantasy point average (short-term)	Exponentially weighted average of last ~5 games, shifted 1 game forward
FANTASY_PTS_EWMA10	Player's recent fantasy point average (long-term)	Exponentially weighted average of last ~10 games, shifted 1 game forward
PTS / REB / AST / STL / BLK / TOV / FG3M _EWMA5 & _EWMA10	Per-stat recent averages	Same EWMA calculation applied to each individual box score stat
MIN_EWMA5 / MIN_EWMA10	Player's recent minutes played	EWMA of minutes played per game, shifted 1 game forward
OPP_DRTG	Opponent team's defensive strength	Team Defensive Rating from Basketball Reference (points allowed per 100 possessions)
OPP_PACE	Opponent team's game pace	Possessions per 48 minutes from Basketball Reference
OWN_ORTG	Player's own team offensive strength	Team Offensive Rating from Basketball Reference
IS_HOME	Whether the game is a home game	1 = home, 0 = away — parsed from the MATCHUP column
IS_B2B	Whether this is a back-to-back game	1 = player played the previous night (days rest = 1), 0 = otherwise
IS_INJURED	Whether the player is on the injury report	1 = player listed as injured in Sleeper API at time of data collection

Results

Fantasy Points distribution

- Right-skewed - most games produce 10 - 30 fpts
- Mean : 23 fpts, but tail end of the distribution can reach 80+ fpts
- High variance is expected
  - Ex. Star player dominant nights vs role player limited minutes



Results - Model Comparison

Test RMSE

- Best Model: Linear Regression Full model
  - Contextual features adds ~0.04 RMSE improvement
- XGBoost and Random Forest do not beat Linear Regression - relationships in data are mostly linear
- Switching from simple rolling averages to EWMA improved every model by 0.08-0.11 RMSE

Model	Features	Test RMSE	CV RMSE
Linear Regression	Baseline	10.177	10.400
Linear Regression	**Full**	**10.136**	**10.354**
XGBoost	Full	10.206	10.514
Random Forest	Full	10.339	10.620

Results - Model Performance

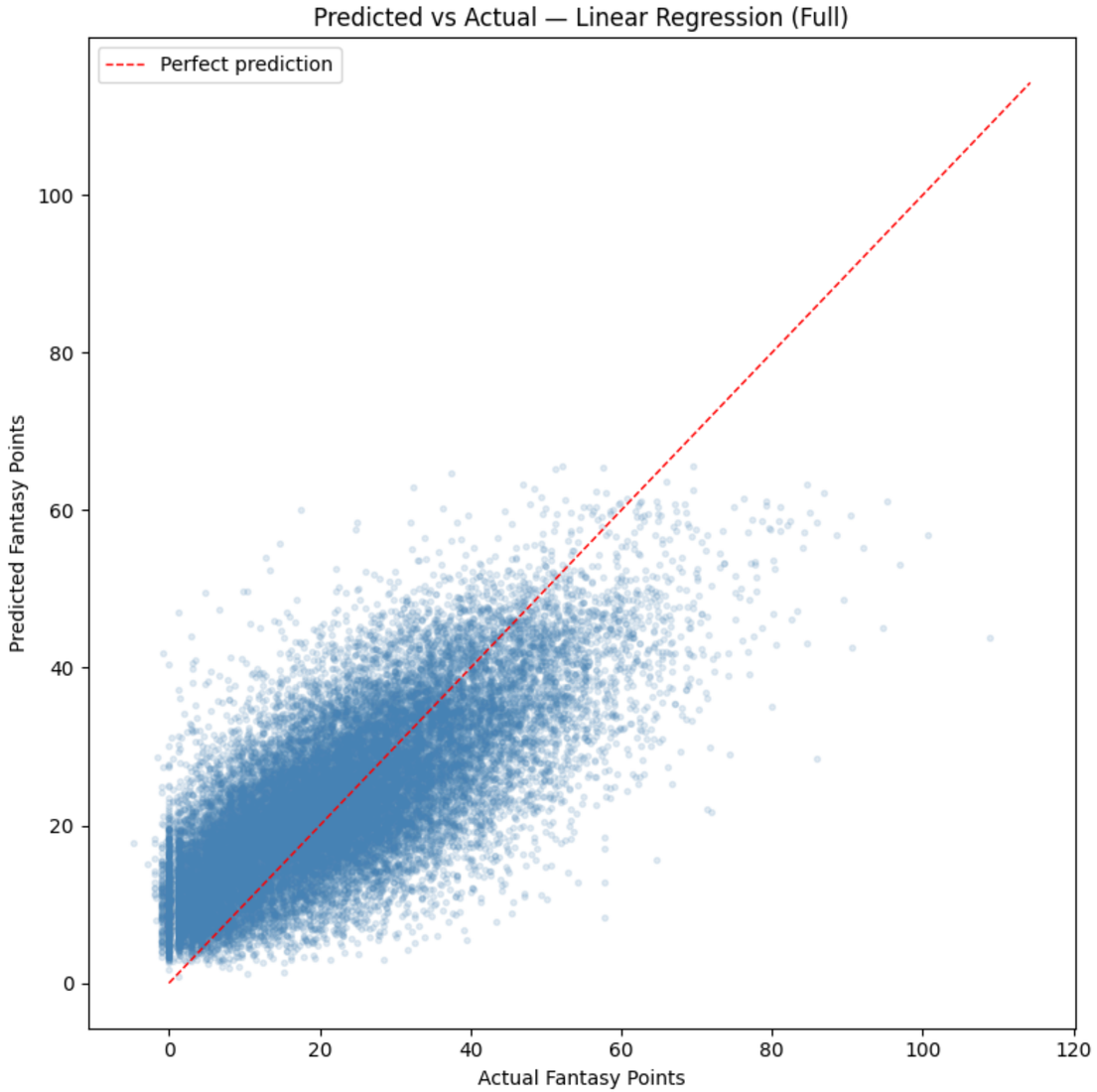
Predicted vs Actual

What the model does well:

- Strong linear correlation in the 10-50 point range - captures typical games accurately

Where it falls short:

- Underestimates extreme high scores (60+)
- Some overestimation near zero - injury scratches or DNPs look normal in recent history
- Remaining error (~10 RSME) reflects genuine unpredictability:
  - foul trouble, blowouts, hot shooting nights



Challenges

- Rate limiting (nba_api): 1.5s delay per request x 530 players x 2 seasons = 30-40min runtime
  - Solution: checkpoint system, creating record CSV per player
- Bot detection (Basketball Reference): Site rejected automated requests.
  - Solution: tuning headers and using **pandas.read_html()**
- Team name mismatch: Basketball Reference uses full team names and nba_api uses abbreviations
  - Solution: manual 30-team mapping dictionary in config.py
- Injury data is a snapshot: Sleeper API only provides current injury status, not historical per-game records.